TUGAS 01

Bab 1: Fondasi Teoretis dan Kerangka Kerja DevSecOps

Matakuliah Workshop DevOps

Dosen Pengampu : Bapak Dr.Ferry Astika Saputra,ST, M.Sc



Oleh :

Nama : Toimul Setyo Andri

NRP : 3126640021

Kelas : D4 LJ TI kelas B

Politeknik Elektronika Negeri Surbaya

Departemen Teknik Informatika dan Komputer

Program Studi Teknik Informatika

September 2026

1. Mengapa DevSecOps tidak dapat direduksi menjadi sekadar penambahan scanner pada pipeline?

Jawaban:

Kalau DevSecOps cuma diartikan sebagai nempelin scanner otomatis di pipeline CI/CD, itu baru menyentuh permukaan teknisnya saja dan belum menyelesaikan akar masalahnya.

Ada beberapa alasan mendasar mengapa pemahaman tersebut kurang tepat:

1. Sistem Sosio-Teknis: DevSecOps itu pada dasarnya adalah sistem yang menggabungkan interaksi manusia, budaya kerja, proses bisnis, arsitektur, hingga kebijakan organisasi. Tool pemindai (scanner) sebagus apa pun tidak akan bisa memperbaiki konflik structural, misalnya kalau tim developer cuma dinilai dari seberapa cepat mereka merilis fitur, sementara tim security dituntut untuk menahan rilis demi ketiadaan risiko.

2. Risiko Penumpukan Temuan (Bottleneck Baru): Tanpa kriteria keputusan yang jelas, scanner hanya bakal memuntahkan ratusan atau ribuan laporan kerentanan. Jika tidak ada mekanisme pemilahan atau triage, penentuan ambang batas atau threshold, serta alur pengecualian (waiver) yang disepakati, tim pengembang justru akan kewalahan atau bahkan sengaja mematikan tool tersebut agar rilis tidak terhambat.

3. Keamanan Harus Hadir di Seluruh Siklus: Scanner di pipeline biasanya baru bekerja pada tahap build atau test (seperti SAST/SCA). Padahal, DevSecOps menuntut kontrol keamanan sejak awal desain (threat modeling pada tahap plan), penerapan secure coding, verifikasi rantai pasok (supply chain), hingga pemantauan saat aplikasi sudah berjalan di lingkungan produksi (runtime detection & observability).

2. Evidence apa yang membedakan klaim kontrol dari kontrol yang benar-benar terverifikasi?

Jawaban:

Klaim kontrol baru sebatas pernyataan bahwa suatu standar telah diterapkan, sedangkan kontrol yang terverifikasi wajib dibuktikan melalui evidence as product, yaitu bukti otentik berbasis data yang dapat diproses oleh mesin (machine-readable) maupun ditinjau oleh manusia, serta dapat diaudit secara independen.

Evidence yang valid dan dapat diverifikasi meliputi:

1. Laporan Pemindaian Terstruktur: Berkas keluaran dari tool keamanan dalam format standar seperti SARIF (untuk analisis statis/SAST) atau JUnit XML.
2. Metadata Rantai Pasok (Software Supply Chain): Dokumen inventaris komponen berupa SBOM (Software Bill of Materials) dengan standar CycloneDX atau SPDX, lengkap dengan identitas digital (digest) dari container image.
3. Jejak Integritas dan Provenansi: Tanda tangan digital (cryptographic signature) pada artefak serta catatan provenance (seperti pada kriteria SLSA) yang membuktikan siapa yang membangun artefak tersebut dan bagaimana proses pembangunannya dilakukan.
4. Dokumentasi Kebijakan dan Keputusan: Catatan evaluasi policy-as-code, log eksekusi pipeline deployment, serta dokumen waiver resmi (yang mencantumkan alasan risiko diterima, pemilik risiko/risk owner, dan batas waktu kedaluwarsa).

Catatan: Screenshot atau pernyataan manual tidak dianggap sebagai evidence yang cukup karena sulit diverifikasi secara otomatis, rawan kehilangan konteks, dan susah untuk ditelusuri kembali (traceability) ke commit Git atau pipeline run tertentu.

3. Bagaimana shared responsibility memengaruhi ownership risiko dan tindak lanjut temuan?

Jawaban:

Prinsip shared responsibility (tanggung jawab bersama) bukan berarti semua orang harus mengerjakan semua hal tentang keamanan, melainkan membagi akuntabilitas secara terstruktur agar tidak ada area abu-abu ketika ditemukan masalah keamanan.

Pengaruhnya terhadap ownership risiko dan tindak lanjut temuan dapat dibagi menjadi beberapa poin:

a. Kejelasan Pembagian Peran (Model RACI):
    1. Product Owner: Memegang ownership terhadap keputusan bisnis dan menetapkan toleransi risiko (risk acceptance).
    2. Developer: Mengambil alih kepemilikan atas kualitas kode, menerapkan secure coding, dan bertanggung jawab meremediasi temuan teknis (seperti melakukan update dependensi atau refactoring).
    3. Platform & Security Engineer: Bertanggung jawab menyediakan jalur build yang aman (golden path), merancang aturan (rule), memfasilitasi threat modeling, serta menyediakan laporan intelijen.
    4. Operations & Governance: Mengelola pengerasan sistem (hardening), deteksi runtime, serta memverifikasi kecukupan evidence untuk kebutuhan audit.

b. Umpan Balik Cepat dan Kolaboratif: Karena tanggung jawab dibagi, temuan scanner tidak lagi dianggap sebagai "kartu merah" dari tim keamanan untuk menyalahkan pengembang. Temuan disajikan sebagai data konteks agar pengembang bisa langsung memperbaikinya saat biaya perubahan masih murah (shift-left).

c. Eskalasi yang Terukur: Jika suatu temuan tidak bisa langsung diperbaiki karena alasan operasional, tindak lanjutnya tidak dilakukan secara ad-hoc. Keputusan untuk menunda perbaikan harus melewati alur eskalasi resmi, di mana risk owner menandatangani dokumen waiver berbatas waktu dengan kompensasi kontrol yang jelas.